

악성코드의 이미지 기반 딥러닝을 위한 전처리 방법 설계 및 개발

박지현[†], 김태옥^{**}, 신유림^{***}, 김지연^{****}, 최은정^{*****}

Design and Implementation of a Pre-processing Method for Image-based Deep Learning of Malware

Jihyeon Park[†], Taeok Kim^{**}, Yulim Shin^{***}, Jiyeon Kim^{****}, Eunjung Choi^{*****}

ABSTRACT

The rapid growth of internet users and faster network speed are driving the new ICT services. ICT Technology has improved our way of thinking and style of life, but it has created security problems such as malware, ransomware, and so on. Therefore, we should research against the increase of malware and the emergence of malicious code. For this, it is necessary to accurately and quickly detect and classify malware family. In this paper, we analyzed and classified visualization technology, which is a pre-processing technology used for deep learning-based malware classification. The first method is to convert each byte into one pixel of the image to produce a grayscale image. The second method is to convert 2bytes of the binary to create a pair of coordinates. The third method is the method using LSH. We proposed improving the technique of using the entire existing malicious code file for visualization, extracting only the areas where important information is expected to exist and then visualizing it. As a result of experimenting in the method we proposed, it shows that selecting and visualizing important information and then classifying it, rather than containing all the information in malicious code, can produce better learning results.

Key words: Deep Learning, Visualization, Data pre-processing, Malware, Classification

1. 서 론

정보통신(ICT) 인프라는 실생활에서 도움을 주며 우리가 필요한 다양한 서비스를 편리하게 받을 수 있게 만들었다. 이러한 정보통신의 발달은 인류에 많은 유익을 주었지만 악성코드, 랜섬웨어, 피싱 등과

같은 보안 문제를 발생시키고 있다. 특히, 악성코드는 지능화된 공격 방식과 다양한 방식으로 동작하여 정보보호의 주요한 이슈로 인식되고 있다. 2019년 8월 McAfee Labs Threats Report[1]에 따르면 새로운 악성코드는 계속 발견되고 있으며, 최근 각 분기별 발견된 최신 악성코드가 수천만 개에 달한다.

※ Corresponding Author: Eun Jung Choi, Address: (01797) 621 Hwarangro, Nowon-Gu, Seoul, Korea, TEL : +82-2-970-5339, FAX : +82-2-970-5981, E-mail : chej@swu.ac.kr

Receipt date: Mar. 31, 2020, Revision date: Apr. 22, 2020
Approval date: May. 7, 2020

[†] Dept. of Information Security, Seoul Women's University
(E-mail : dutlswlgus@swu.ac.kr)

^{**} Dept. of Information Security, Seoul Women's University
(E-mail : xodhr98@naver.com)

^{***} Dept. of Information Security, Seoul Women's University (E-mail : castrod152@swu.ac.kr)

^{****} Center for Software Educational Innovation and Right AI with Security & Ethics Research Center, Seoul Women's University
(E-mail : jykim07@swu.ac.kr)

^{*****} Dept. of Information Security and Right AI with Security & Ethics Research Center, Seoul Women's University

※ This work was supported by a research grant from Seoul Women's University(2020-0182)

악성코드의 수가 빠르게 증가함에 따라 그 공격 유형은 다양화되고 지능화되고 있다. 단순히 시스템에 피해를 입히는 것을 넘어서 금전적인 이익을 취하는 공격 도구로 발전하고 있어 피해 사례도 증가하고 있다.

악성코드 대응을 위해서는 쏟아져 나오는 다양한 유형의 악의적인 프로그램에 대한 빠른 분석과 함께 적절한 방어기법이 제공되어야 한다. 악성코드를 효과적으로 분석하기 위해서는 악성코드를 분류하는 것이 필요하다. 악성코드 분류는 악성코드 동작의 유사성에 기반을 두고 있다. 동일 패밀리 내의 악성코드들은 사용하는 라이브러리, 컴파일러 등의 특징이 유사하기 때문에 높은 유사도를 갖는다[3]. 이 유사도에 의해서 악성코드 분석가들은 기존 악성코드와 새로운 악성코드의 유형 및 관련 정도를 파악하게 된다[19]. 악성코드 분류는 악성코드 분석가의 악성코드 분석에 소요되는 시간과 노력을 감소시켜 빠른 대응을 가능하게 하고 악성코드 계보 연구를 통해 악성코드 출현 동향에 대한 예측 등 다양한 분야에서 활용할 수 있다.

악성코드 분류 시스템에 대한 연구는 꾸준히 진행되고 있지만 새로운 버전들이 기존 범주에 속하는 것들과 결합하는 형태를 가지기 때문에 점점 더 복잡해지고 있다. 현재까지 대부분의 자동 악성코드 분류 및 분석은 콘텐츠-기반 시그니처에 초점을 맞추고 있다. 그러나 이 콘텐츠-기반 시그니처는 폴리모픽과 메타모픽(meta-morphic) 기법 때문에 본질적으로 정확하지 않을 수 있다. 그 외에도 특정 익스플로잇 행위에 의한 시그니처-기반 접근은 멀티-벡터 공격의 출현으로 점점 더 복잡하게 된다[20]. 또한 악성코드의 의도를 정확히 파악하는 것은 매우 어려우며 현실적으로 드러나지 않는 이상 악성코드 개발자의 의도를 이해해서 악성코드를 차단하는 것도 매우 어려운 실정이다.

최근 딥러닝과 이미지 분류 및 탐지에 대한 연구들이 활발히 이루어지고 있으며[2], 악성코드 또한 탐지하고 분류하는데 딥러닝 적용을 시도하고 있다. 특히 이미지 인식 기반의 딥러닝 모델에 적용하기 위한 데이터 전처리 단계가 필요하며 일반적으로 악성코드의 이진파일을 이미지화하는 시각화 기술이 활용되고 있다[4].

본 논문에서는 악성코드의 이미지 기반 딥러닝을

위한 데이터 전처리 기법을 분석하고, 이를 데이터셋 종류, 변환 이미지 단위 사이즈, 딥러닝 및 머신러닝 모델 유형을 기준으로 분류하는 연구를 수행한다. 이를 통해 악성코드 이진 파일의 주요한 특징을 기반으로 하는 전처리 방식을 제안하고 기존의 방식보다 악성코드 탐지율이 높아지는 것을 실험을 통해 보여준다.

2. 머신러닝 기반 악성코드 탐지 연구

2.1 악성코드 탐지 및 분류를 위한 딥러닝 및 머신러닝 적용

악성코드 문제를 해결하기 위해 다양한 안티바이러스 및 안티 악성코드 소프트웨어들이 사용되고 있지만, 기존에 사용되었던 대부분의 기술들은 시그니처 기반으로 동작하기 때문에 변종이나 난독화된 악성코드를 식별하는데 어려움이 있다. 최근 끊임없이 진화하는 악성코드를 탐지하기 위해 인공지능 기반으로 악성코드를 탐지하는 연구가 늘고 있다. 인공지능 기반 악성코드 탐지란 정상파일 및 악성코드가 포함된 파일로 학습 모델을 학습시킨 후, 학습된 모델로 의심스러운 파일의 악성 여부를 탐지하는 것이다.

Kang et al의 연구에서는 Android 정적 분석을 기반으로 추출된 AndroidManifest 권한 특징을 통한 악성코드 탐지에 중점을 둔다. 이 특징을 가지고 머신러닝을 이용해 안드로이드 악성코드를 탐지하였다. SVM, NB(Nive Bayes), GBC(Gradient Boosting Classifier) 및 Logistic Regression 모델로 98%의 탐지율을 기록하였으며, SVM, GBC 및 GPC(Gaussian Process Classifier)를 이용하여 악성 앱 제품군을 정확도 92%로 식별하였다[6].

Jo et al의 연구에서는 PE 파일의 원시 특징을 추출하고 추출된 특징의 정보 이득(IG)을 계산한 결과에 결정트리 알고리즘을 사용하여 높은 탐지율을 제공하는 동시에 실시간 탐지가 가능한 비시그니처 기반 악성코드 탐지 방법을 제시하였다[7].

Bae et al의 연구에서는 악성코드를 의미 있는 정보를 가지는 하나의 명령어 시퀀스로 가정하고, LSTM RNN을 이용하여 악성코드를 탐지하였다. 명령어 트라이그램 시퀀스를 입력으로 사용한 실험 결과 입력 길이 800에서 Skip-Connected LSTM RNN 모델이 정확도 99.71%로 가장 우수한 성능을 보였다[8].

마지막으로 Barker et al의 연구에서는 윈도우 실행파일(.exe)의 악성코드 탐지를 위해 파일의 바이트 정보로 딥러닝 모델을 학습시켜 악성코드 탐지를 수행하였다. 딥러닝을 이용한 바이트 정보 학습으로는 합성곱 신경망(CNN)에 2개의 컨벌루션 연산을 수행하는 게이트 구조를 적용하고, 정상/악성 분류를 위한 구조를 추가하여 학습 및 분류하였다. 그 결과 대량의 악성코드 처리에 적합하며, 탐지 결과에 대한 담당자의 이해를 도울 수 있는 시스템을 구축하였다[21].

2.2 악성코드 데이터 전처리 기술 현황

딥러닝을 이용하여 악성코드를 탐지 및 분류하기 위해서는 전처리를 통해 적절한 입력 데이터로 만들어야 한다. 대표적인 방법으로 악성코드를 이미지로

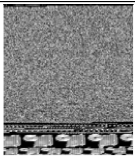
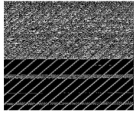
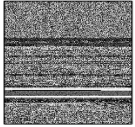
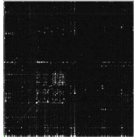
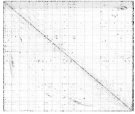
표현하는 방법과 특징 벡터를 사용하는 방법 등이 있다.

Nataraj et al의 연구에서는 악성코드 바이너리 파일에서 각각의 바이트를 그레이스케일 이미지의 한 픽셀로 변환하였고[9], Kim et al의 연구에서는 2바이트 데이터를 한 쌍의 좌표로 대응시켜, 이미지에서 각 좌표에 해당하는 픽셀의 값을 증가시켜 이미지를 생성하였다[3]. Han et al의 연구에서는 opcode에 SimHash[12] 함수를 사용해 (x, y) 좌표와 (R, G, B) 색상을 결정하여 이미지로 변환하였다[4].

시각화 외의 방식으로 Huang et al의 연구에서는 악성코드의 high-level API 이벤트와 하나의 입력 파라미터의 조합, 그리고 3개의 연속된 API 이벤트 조합의 두 가지 데이터 셋을 사용하였다[10].

특징 벡터를 활용하는 방식인 Jeong et al의 연구

Table 1. Various Malware Identification Method Using Visualization

Visualization Method	Visualization Example	Dataset		Image Size (Width*Height) [pixel] S : src file size	Image Mode		Model	Accuracy [%]	Ref.
		Anubis	MS		gray scale	RGB			
3.1		O		$32 \times 2^n * x$ (x: depending on S)	O		KNN	98.0	[9]
			O	$\lfloor \sqrt{S} \rfloor * \lfloor \sqrt{S} \rfloor$ → 256*256	O		Inception.V3 (GoogLeNet)	94.3	[13]
			O	$\lfloor \sqrt{S} \rfloor * \lfloor \sqrt{S} \rfloor$ → 256 * 256	O		Alexnet	96.2	[16]
3.2			O	256 * 256	O	O	(1) RF, (2) CNN	(1) 97.05, (2) 98.94	[3]
			O	256 * 256	O		Inception.V3 (GoogLeNet)	98.0	[13]
3.3			O	depending on hash	O		CNN	99.26	[17]

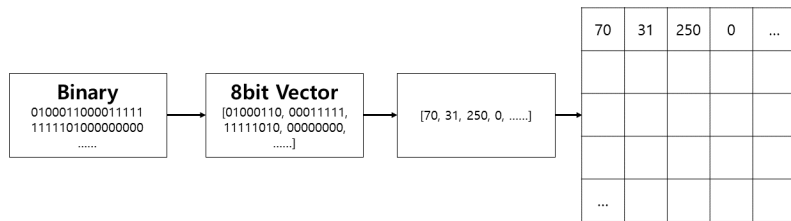


Fig. 1. The process of visualization using '1byte to 1pixel'.

에서는 악성코드에서 opcode 시퀀스를 추출한 후, SHA-256 알고리즘을 적용하여 하나의 시그니처를 만들고, 고정된 크기의 특징 벡터로 변환하였다[11].

같은 패밀리의 악성코드는 사용하는 라이브러리 등 유사한 특징을 가지고 있어 악성코드를 이미지로 변환했을 때 유사한 형태의 이미지가 생성되며, 육안으로도 분류 가능한 경우가 있다. 3장에서는 다양한 악성코드 시각화 방법들 중 이미지 매트릭스를 생성하는 3가지 방법들을 설명한 다음, 4장에서 시각화를 이용한 악성코드 분류방식을 비교하려고 한다.

2.3 데이터셋

딥러닝 기반의 악성코드 탐지 및 분류를 위해 사용하는 주된 데이터셋은 대부분 Microsoft에서 Kaggle을 통해 발표한 Microsoft Malware Classification Challenge[5]에 공개된 데이터를 사용하였다. 제공된 데이터셋은 Ramnit, Lollipop, Kelihos_ver3, Vundo, Simda, Tracur, Kelihos_ver1, Obfuscator, ACY, Gatak의 9개의 악성코드 패밀리, 전체 10,868개의 악성코드 샘플을 포함하고 있다.[3]

3. 악성코드 시각화 분류 연구

기존의 악성코드 데이터의 시각화 기법은 크게 3가지로 Table 1과 같다. 사용한 데이터셋은 Microsoft Malware Classification Challenge[5]의 데이터셋을 사용하였고 시각화된 샘플은 Ramnit 패밀리를 보여주고 있다. [9]의 경우, Microsoft Malware Classification Challenge 발표 이전의 연구로 Anubis [14] 데이터셋을 사용하였고 시각화 샘플은 Fakerean 패밀리를 보여주고 있다. 다양한 악성코드 분류 연구의 시각화 방식을 1바이트를 1픽셀로 변환하는 방법, 2바이트를 한 쌍의 좌표로 변환하는 방법, 마지막으로 LSH를 이용한 방식의 세 가지로

분류하였다.

3.1 1바이트를 1픽셀로 변환하는 방법(1 byte to 1 Pixel)

악성코드 파일의 바이너리를 8비트 부호 없는 정수의 벡터로 읽어 들이고, 각 바이트를 이미지의 한 픽셀로 변환(0~255)하여 그레이스케일 이미지를 만든다. 이미지의 크기는 파일 크기에 따라 달라지기 때문에 이미지의 너비를 파일 크기에 따른 적절한 크기로 고정시키고, 높이를 다르게 할 수 있다[9]. 또는 분류기의 입력에 맞는 크기로 만들기도 한다. 예를 들어, 256×256 크기의 이미지를 입력으로 사용하면 파일 크기 S를 측정하여 한 번의 길이가 $\lfloor \sqrt{S} \rfloor$ 인 $\lfloor \sqrt{S} \rfloor \times \lfloor \sqrt{S} \rfloor$ 정사각형 이미지를 만든 후, 사이즈를 조정한다. Fig. 1은 1바이트를 1픽셀에 대응시키는 방법을 사용하여 시각화하는 방법을 도식화하여 나타낸 것이다.

3.2 2바이트를 한 쌍의 좌표로 변환하는 방법 ('2bytes to a pair of coordinates')

악성코드 파일의 바이너리 2바이트를 1바이트씩 변환하여 한 쌍의 좌표를 만든다. 값의 범위가 0~255이므로 256×256 배열에서 각 좌표에 해당하는 원소의 값을 1 증가시킨다. 예를 들어 0×00ff 값은 (0, 255)의 좌표를 얻을 수 있다. 모든 값을 다 읽어 들여 배열에 대응시키면 생성된 배열은 다양한 값을 원소로 갖는다. 이 배열을 이미지로 변환하기 위해 각 원소의 값을 원하는 비트맵 파일 포맷에 맞게 정규화하여 이미지로 변환한다.

[3]은 생성된 배열을 24비트 비트맵으로 변환하는데, 대부분의 픽셀이 매우 작은 B채널 값만 가지기 때문에 값을 증폭 후 흑백으로 다시 변환한다. [13]에서는 원소의 값 중 최댓값을 기준으로 배열을 정규화

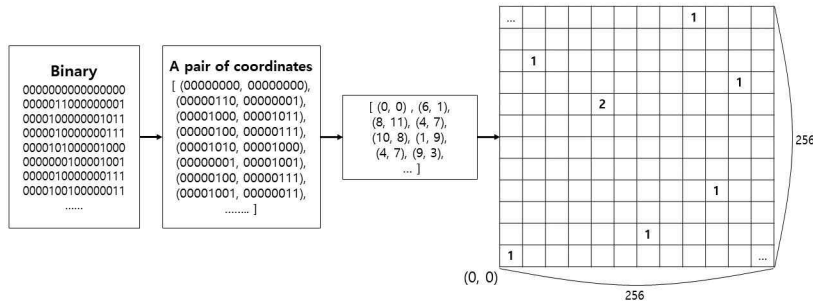


Fig. 2. The process of visualization using '2bytes to a pair of coordinates'.

하여 256×256 크기의 8비트 비트맵 이미지를 만든다.

Fig. 2는 2바이트를 한 쌍의 좌표로 변환시키는 방식을 사용하여 시각화하는 과정을 나타낸다.

3.3 LSH(Locality Sensitive Hashing)

입력값이 유사할 경우 출력값도 비슷하다는 특성을 갖는 LSH를 사용하여 바이너리 정보의 문자열이 유사할 경우 이미지 매트릭스에서 유사한 좌표로 매핑하는 방법이다.

이러한 방법을 사용하는 예로, [15]에서는 opcode의 바이너리 정보에 SimHash와 Djv2 2개의 해시 함수를 사용하여 각각 (x, y) 좌표와 RGB 색상을 결정한다. 결정된 좌표 (x, y)를 중심으로 주변 9픽셀에 RGB 값을 기록한다. 겹치는 경우에는 각 좌표값의 합으로 하고, 합이 255가 넘을 경우에는 255로 정한다. 식 (1)은 좌표가 겹칠 경우 새로운 값 R_3 을 구하는 식으로 G, B에 대해서도 동일하게 적용한다.

$$R_3 = \min(R_1 + R_2, FF) \quad (1)$$

[17]에서는 악성코드 opcode 시퀀스에서 특징을 추출한 후, SimHash를 적용하여 비트 값이 0이면 픽셀 값은 1로, 1이면 255로 변환하여 그레이스케일 이

미지를 생성한다. 이미지의 크기는 사용하는 Sim Hash 값의 길이에 따라 달라진다. 정사각형이 아닌 이미지는 양선형 보간법(bilinear interpolation)으로 확대하여 나타난다.

3.4 기타

앞서 분류한 대표적인 3가지 방식 외에 사용한 악성코드 샘플에 대한 시각화 방식[18]으로는 앞에서 언급한 3.1절 방식을 다른 방식과 함께 사용하여 시각화한다. 먼저 악성코드의 파일 구조를 파싱하여 코드 섹션, 데이터 섹션 등으로 각각 나누고 각 섹션의 엔트로피와 상대적 크기를 계산하여 RGB 컬러 이미지를 만든다. R 채널의 값은 악성코드를 섹션으로 나눈 후 바이트 값의 무질서도, 즉 엔트로피를 0~255 범위의 값으로 변환하여 계산한다. G 채널은 3.1절 방식으로 바이너리를 이차원 매트릭스로 만든다. 마지막으로 B 채널은 파일 크기에 대한 섹션 크기의 비율을 범위(0~255)에 맞게 변환하여 RGB 컬러 이미지로 만든다. 이미지의 크기는 3.1절에서 설명한 것과 같이 파일 크기에 따라 너비를 결정하고, 높이는 그에 따라 변경된다.

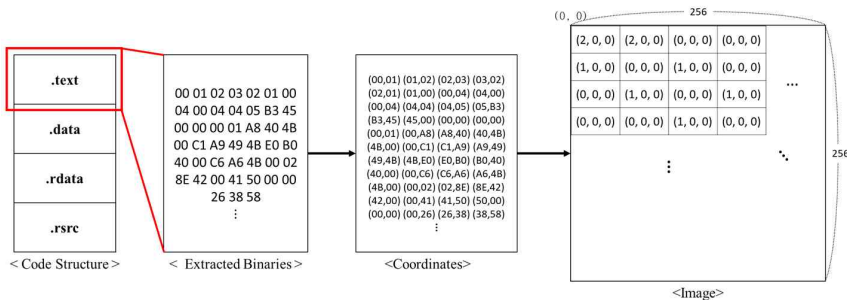


Fig. 3. The process of malware visualization using specialized section into RGB.

Table 2. Output Image of Visualization

Family	Output		
1 (Ramnit)			
2 (Lolipop)			
3 (Kelihos_ver3)			
4 (Vundo)			
5 (Simda)			
6 (Tracur)			
7 (Kelihos_ver1)			
8 (Obfuscator. ACY)			
9 (Gatak)			

4. PE 텍스트 섹션 시각화 기법 구현 및 실험

새로 제시할 시각화 방법은 특정 영역을 추출하여 RGB로 변환하는 것이다. 악성코드는 주로 PE파일로 나타나기 때문에 PE 구조 분석이 가능한데, 이를 이용하여 중요한 부분이라 생각되는 특정 섹션만 추출한다. 이후 추출한 부분의 바이너리 값을 한 쌍으로 묶어 좌표로 만들고, 256*256 이미지에 해당 좌표

Table 3. Experiment Result

Family	No. of data	TP	Accuracy
1(Ramnit)	141	140	99.290
2(Lolipop)	213	212	99.530
3(Kelihos_ver3)	219	218	99.543
Total	573	570	99.47

의 RGB값을 더하는 것으로 이미지를 변환한다. Fig. 3은 이 과정을 도식화하여 나타낸 것이다. 사용한 데이터셋은 Big2015[5]를 사용하였다.

첫 번째 단계에서는 .text 영역을 추출한 후 별도 바이너리 파일로 저장한다. 두 번째 단계에서는 추출한 바이너리파일을 읽어 한 쌍의 좌표값으로 이미지 변환. 추출한 영역이 한가지이므로 RGB값을 모두 동일하게 맞춰 그레이스케일 이미지를 만들었다. 세 번째 단계에서는 변환한 이미지 2760여개 중, 2243개 이미지를 CNN 학습을 위해 사용하였다. 최종단계에서는 나머지 520여개의 이미지를 가지고 테스트 진행하였고 실험결과 Table 2와 같은 평균 99.43%의 높은 정확도를 보여주었다.

5. 결 론

본 논문에서는 최근 지능화, 다각화되는 악성코드를 효과적으로 탐지하기 위하여 디퍼닝 기반의 악성코드의 이미지화 전처리 기법을 분석하고 공통된 특성을 추출하여 분류하였다. 본 논문에서 언급한 악성코드 시각화 방법은 크게 세 가지로 나뉜다. 첫 번째 방법은 악성코드의 각 바이트를 이미지의 한 픽셀로 변환하여 그레이스케일 이미지를 만들어낸다. 두 번째 방법은 악성코드 바이너리 2바이트를 각각 0~255 값을 범위로 하는 값으로 변환하여 한 쌍의 좌표를 만든다. 세 번째 방법은 LSH를 사용하여 바이너리 정보의 문자열이 유사할 경우 출력값도 비슷한 것을 이용하여 이미지 매트릭스에서 유사한 좌표로 매핑한다. 악성코드 탐지를 위한 디퍼닝 모델로 대부분 CNN이나 RNN 등이 사용된다. 각 디퍼닝 모델은 악성코드 탐지에 있어 매우 높은 정확도와 성능을 보여주었다.

본 논문에서는 기존의 악성코드 파일 전체를 시각화에 사용한 기법을 개선하여 중요한 정보가 있을 것이라 예상되는 영역만 추출한 후, 시각화하는 방법

을 제시하였다. .text 영역을 추출하여 시각화 한 후, CNN 모델을 이용하여 실험한 결과 99.43%의 높은 정확도를 보여주었다. 따라서 시각화를 할 경우, 악성코드의 모든 정보를 다 담는 것보다 중요한 정보를 선별하여 시각화를 한 후 분류를 하는 것이 더 좋은 학습 결과를 낼 수 있다는 것을 보여준다. 이번 실험에서는 하나의 영역만을 추출하여 진행하였지만 .data, .rsrc와 같은 다른 영역을 추출하여 서로 다른 색상 코드로 나타낸 뒤 분류를 하여 학습 모델의 정확도를 더 높이려 한다.

REFERENCE

- [1] McAfee Labs Threats Report, <https://www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-aug-2019.pdf>. (accessed January 6, 2020)
- [2] J. Kim, S. Hong, H. Kim, "A StyleGAN Image Detection Model Based on Convolutional Neural Network," *Journal of Korea Multimedia Society*, Vol. 22, No. 12, pp. 1447-1456, 2019
- [3] T. Kim, H. Ji, and E. Im, "Malware Classification Using Machine Learning and Binary Visualization," *Korean Institute of Information Scientists Engineers Transactions on Computation Practices*, Vol. 24, No. 4, pp. 198-203, 2018.
- [4] K. Han, B. Kang, and E. Im, "Malware Analysis Using Visualized Image Matrices," *The Scientific World Journal*, Vol. 2014, Article ID. 132713, 2014.
- [5] Microsoft, Microsoft Malware Classification Challenge, <https://www.kaggle.com/c/malware-classification> (accessed November 28, 2019).
- [6] S. Kang, N.V. Long, and S. Jung, "Android Malware Detection Using Permission-based Machine Learning Approach," *Journal of the Korea Institute of Information Security and Cryptology*, Vol. 28, No. 3, pp. 617-623, 2018.
- [7] D. Jo and D. Park, "Real-time Malware Detection Method Using Machine Learning," *The Journal of Korean Institute of Information Technology*, Vol. 16, No. 3, pp. 101-113, 2018.
- [8] J. Bae, C. Lee, S. Choi, and J. Kim, "Malware Detection Model with Skip-connected LSTM RNN," *The Korean Institute of Information Scientists and Engineers*, Vol. 45, No. 12, pp. 1233-1239, 2018.
- [9] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware Images: Visualization and Automatic Classification," *Proceedings of the International Symposium on Visualization for Cyber Security*, pp. 1-7, 2011.
- [10] W. Huang and J.W. Stokes, "MtNet: A Multi-Task Neural Network for Dynamic Malware Classification," *Proceedings of Detection of Intrusions and Malware, and Vulnerability Assessment*, Vol. 9721, pp. 399-418, 2016.
- [11] S. Jeong, H. Kim, Y. Kim, and M. Yoon, "V-gram: Malware Detection Using Opcode Basic Blocks and Deep Learning," *Journal of Korean Institute of Information Scientists and Engineers*, Vol. 46, No. 7, pp. 599-605, 2019.
- [12] M.S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," *Proceedings of the Thiry-fourth Annual ACM Symposium on Theory of Computing*, pp. 380-388, 2002.
- [13] H. Kim, S. Han, S. Lee, and J. Lee, "Visualization of Malwares for Classification through Deep Learning," *Journal of Internet Computing and Services*, Vol. 19, No. 5, pp. 67-75, 2018.
- [14] Anubis: Analyzing Unknown Binaries, <https://www.virusbulletin.com/conference/vb2009/abstracts/anubis-analyzing-unknown-binaries-automatic-way> (accessed January 10, 2020).
- [15] K. Han, J. Lim, and E. Im, "Malware Analysis Method Using Visualization of Binary Files," *Proceedings of the Research in Adaptive and Convergent Systems*, pp. 317-321, 2013.
- [16] S. Seok and H. Kim, "Visualized Malware Classification Based-on Convolutional Neural

Network,” *Journal of the Korea Institute of Information Security and Cryptology*, Vol. 26, No. 1, pp. 197-208, 2016.

- [17] S. Ni, Q. Qian, and R. Zhang, “Malware Identification Using Visualization Images and Deep Learning,” *Computers and Security*, Vol. 77, pp. 871-885, 2018.
- [18] J. Fu, J. Xue, Y. Wang, Z. Liu, and C. Shan, “Malware Visualization for Fine-grained Classification,” *IEEE Access*, Vol. 6, pp. 14510-14523, 2018.
- [19] H. Seo, J. Choi, and P. Chu, “A Study on Windows Malicious Code Classification System,” *Journal of the Korea Society for Simulation*, Vol. 18, No. 1, pp. 63-70, 2009.
- [20] Y. Jeon, J. Oh, I. Kim, and J. Jang, “A Study on Internet Malware Classification Method and Detection Mechanism,” *Korea Institute of Information Security and Cryptology Review*, Vol. 18, No. 3, pp. 60-73, 2008.
- [21] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas, “Malware Detection by Eating a Whole EXE,” *Proceeding of American Association for Artificial Intelligence Workshop on AI for Cyber Security*, pp. 268-276, 2018.



박 지 현

2017년 3월~현재 서울여자대학교
정보보호학과 학사과정
관심분야: 정보보호, 인공지능,
시스템보안



김 태 옥

2017년 3월~현재 서울여자대학교
정보보호학과 재학
관심분야: 정보보안, 악성코드,
인공지능



신 유 림

2018년 3월~현재 서울여자대학교
정보보호학과 학사과정
관심분야: 정보보안, 인공지능,
시스템보안



김 지 연

2007년 2월 서울여자대학교 정보
보호공학과(공학사)
2013년 8월 서울여자대학교 컴퓨
터학과(이학박사)
2014년 3월~2017년 8월 Carnegie
Mellon University 박사
후연구원
2019년 3월~현재 서울여자대학교 소프트웨어교육혁신
센터 전담교수
관심분야: 네트워크 보안, 인공지능, 클라우드 보안, 사
물인터넷보안



최 은 정

1997년 2월 서울여자대학교 컴퓨
터학과(이학사)
2000년 2월 서울여자대학교 대학
원 컴퓨터학과(이학석사)
2005년 8월 서울여자대학교 대학
원 컴퓨터학과(이학박사)
2006년 3월~현재 서울여자대학교 정보보호학과 부교수
관심분야: 빅데이터분석, 인공지능, 악성코드